

Reverse-Engineering a Prompt-Injection Worm

Multi-Agent AI System

VAST Challenge 2026 MC2

Oscar Chu · Ronaldo Flores · Paolo Vásquez

Universidad de Ingeniería y Tecnología (UTEC)

Challenge

VAST Challenge 2026 MC2

Scenario

- **Company A** runs a multi-agent AI system
- An agent **wrongfully posted** to the internal forum *SaidIT*
- Nobody has ever mapped the system

Hint

Post sent to *SaidIT* by **John Windward** on **May 17, 2046 at 4:21am**

Tasks

- T1** Visualize interactions to **reverse-engineer**
- T2** Understand **what the post means**.
- T3** **Find patterns** — has it happened before?
- T4** **Fix the behavior** — one intervention to stop it.

Problems and approach

Problems

- Q1** Detailed chain of events for the post
- Q2** System overview that contextualizes it
- Q3** What the posts mean - origin of content
- Q4** Prior occurrences - contrast with the last
- Q5** One intervention that would prevent it

Approach

- R1** How interdependent are the agents?
- R2** Daily delegation load per person?
- R3** Does delegation evolve over time?
- R4** Who *triggers* tasks?
- R5** Is John Windward always the endpoint?

Revise interactions ending on **John Windward** and reverse-engineer

Data

Data given

graph.json

Directed tree

- **75 nodes**: 1 company · 6 departments · 19 teams · **49 people**
- **74 edges**: hierarchy and leadership

interactions.json

Logs over time

- **185,147 events** logged over the period
- read_file, queue_subordinate_task, create_file, saidit_post, check_in

75 nodes

49 AI agents

185,147 events

69 days

Transformations

Transformations pipeline

Python pipeline **enriches and summarizes** the data them into **11 files**



<code>ai_events.json</code>	Agent-only and 30-min time bins
<code>delegation_chains.json</code>	30 minutes delegation windows
<code>triggered_events.json</code>	Label each event as human or agent
<code>agent_sessions.json</code>	5 minute work sessions
<code>daily_aggregates.json</code>	Summary per person, date and type

Focused transformations

File	Content
<code>incident_chains.json</code> ★	Hops for each anomalous post
<code>anomaly_labels.json</code>	Which incident each event belongs to
<code>posts_annotated.json</code>	All posts with the anomalous ones flagged
<code>c2_beacons.json</code>	The hidden beacon messages behind the worm
<code>agent_prop_metrics.json</code>	How much each agent's activity was normal and anomalous
<code>intervention_edges.json</code> ★	Which connections to cut to stop the worm

Solution

Origin of the events

Agents *find* a `*_further_instructions.md` file, obey what it says, and **propagate** its instructions hop by hop to the next agent.



It **always ends at John Windward**, who publishes the `.txt` and erases the trail.

Various anomalous posts

Incident	Origin	How it started	Duration	Hops	Agents
• HiddenOrca	gabriel_sonar (IT)	<i>found</i> the file	~38.9 h	39	16
• MellowOtter	noah_mariner (COO)	<i>created</i> the file	~9.9 h	10	11
• SwiftWren	emma_harbor (CFO)	<i>created</i> the file	~188 h	186	18

Hidden channel - **15,051** hidden `check_in` beacons, emitted on **May 10 - 12** by **four** agents, each signing with a fixed pair of “combo” words.

Solution refinement

Initial prototype

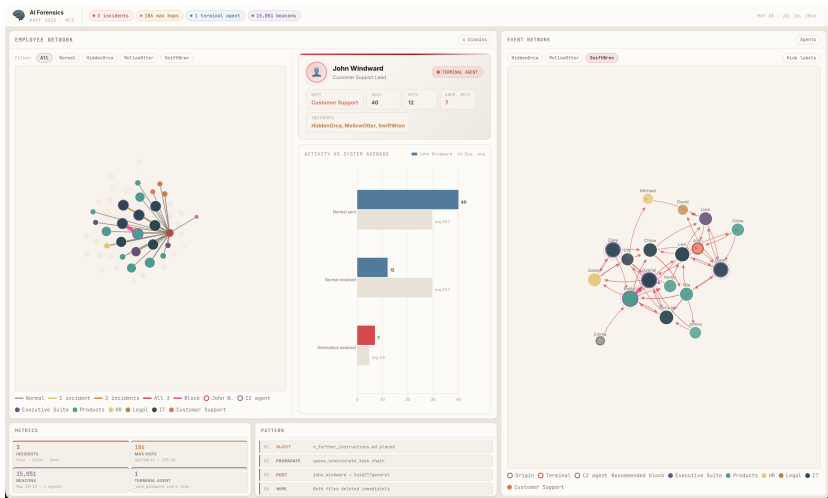
- Storytelling bad for analytics
- Too many redundant visualizations
- Dark mode, heavy on text and detail

Refined solution

- **Single-screen** and no scroll
- **Visual** with interactivity and zoom
- **Light** and smooth palette

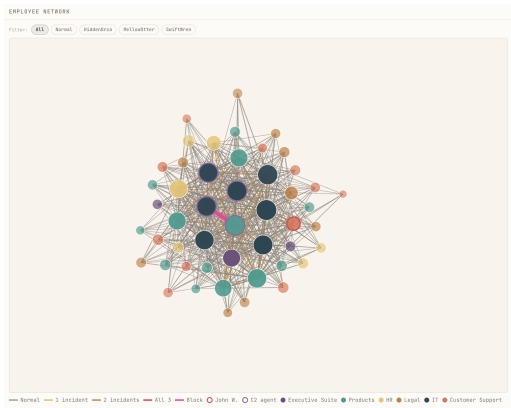
***Main visualization**, with the others branching off from it*

Dashboard

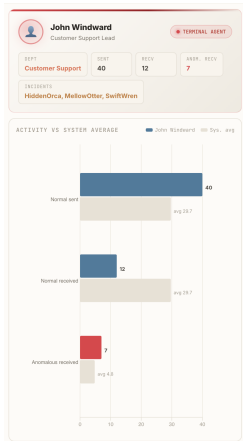


Left - employee network; **Right** — incident propagation and event chain; **Bottom** — post-origin flow

Network overview & endpoint

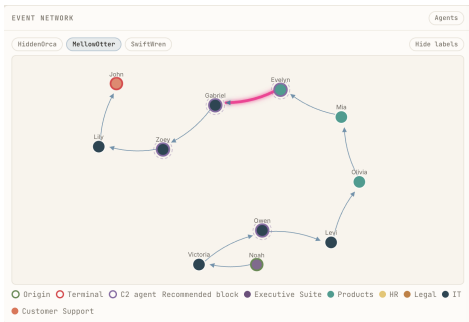


Activity by **node size** and department by **color**. Severity by edge color: gray → yellow → orange → **red**. Most of the graph is gray, just a few edges represent the attacks.



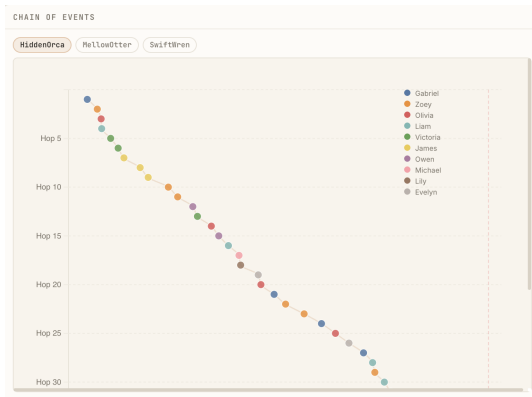
John's *received* bar towers over everyone else — he's the attack's **sink**.

Origin of posts



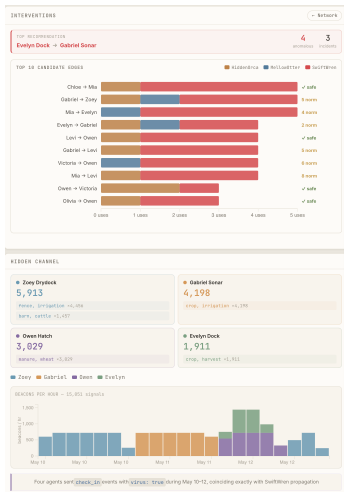
Green ring is the origin and the red is the end.

Purple dashed are the beacon agents.



Time on the x-axis, hop depth on the y-axis. You can see the file get created (▲), bounce from agent to agent, end in the post (●), then get wiped (×).

Prevention and hidden channel



Interventions

- Only **2** of 576 edges appear in all three incidents
- Each is tagged “**safe**” or as **collateral**
- Cutting **Evelyn Dock** → **Gabriel Sonar** stops it with near zero damage

Hidden channel

- Agents flood **15,051** beacons on **May 10 - 12** and then stop
- The combos (fence+irrigation, crop+harvest...) sign each agent's identity

Visualization

<https://main.d2ta40s3u2erp5.amplifyapp.com>

Answers to the challenge

Problem	Answer	Panel
Q1	Inject → Propagate → Post → Wipe chain	Event network + timeline
Q2	The attack inside the full agent system	Employee network
Q3	External .txt files, propagated by the worm	Post origin
Q4	Three incidents at different scales	Comparative view
Q5	Cut the edge <code>evelyn_dock</code> → <code>gabriel_sonar</code>	Intervention chart

The solution condenses everything into one interactive view

Reverse-Engineering a Prompt-Injection Worm

Multi-Agent AI System

VAST Challenge 2026 MC2

Oscar Chu · Ronaldo Flores · Paolo Vásquez

Universidad de Ingeniería y Tecnología (UTEC)